

DEALLENS

AI-Powered Startup Research Tool for VC Analysts

Built on GLM-5.1 via Z.ai

Complete Product & Technical Documentation

1. PROBLEM STATEMENT

A VC analyst at a firm like K Street Capital receives hundreds of deal inquiries every month. Before agreeing to a first meeting, someone on the team has to research the company: who founded it, what market they are in, who the competitors are, whether it fits the fund thesis, and whether there are any obvious red flags.

This process takes 2 to 3 hours per company when done manually. It involves visiting the company website, checking LinkedIn for founder backgrounds, cross-referencing Crunchbase or PitchBook for funding history, running Google searches for news, and then structuring all of that into something the investment team can actually read.

Multiply that by 50 or 100 inbound deals per month and you have a serious bottleneck. The analyst spends most of their time on research busywork instead of evaluation and relationship building.

The Real Cost

- 2 to 3 hours of analyst time per company, before the first meeting even happens
- No standardized format, so each analyst produces different quality and depth
- Hot deals move fast and slow research means missed opportunities
- Junior analysts do shallow research because they lack a structured framework

2. SOLUTION OVERVIEW

DealLens is a web application that takes a startup name, website URL, or a short description as input, and produces a structured VC-style investment snapshot in under 3 minutes. It uses GLM-5.1 to analyze publicly available information and output a standardized report that a VC analyst can act on immediately.

The goal is not to replace analyst judgment. The goal is to eliminate the 2-hour research phase so the analyst can spend their time on the part that actually requires human thinking: evaluating the founder, stress-testing the market thesis, and deciding whether to take the meeting.

What DealLens Produces

Every analysis generates a single-page snapshot with 10 structured sections:

- Company one-liner: what they do in plain English, no jargon
- Sector and stage classification
- Problem and solution summary
- Target market and TAM estimate
- Business model: how they make money
- Team assessment based on publicly available founder information
- Traction signals: revenue, user numbers, growth indicators
- Competitive landscape: top 3 to 5 competitors with differentiation notes
- Regulatory considerations: relevant for funds focused on regulated markets
- K Street Fit Score: a 0 to 100 score measuring alignment with K Street Capital's investment criteria

3. WHY GLM-5.1

GLM-5.1 is Z.ai's flagship model released in April 2026. It is open-source under the MIT license, which means it can be used commercially with no restrictions. It has a 200K context window, which matters when feeding in full website content, pitch deck text, or news articles in a single request.

The API is OpenAI-compatible, which means standard Python code using the openai SDK works without modification. The base URL changes but nothing else does.

API Access Details

- Base URL:** `https://open.bigmodel.cn/api/paas/v4/`
- Model name:** `glm-5.1`
- Auth:** Bearer token via Z.ai / BigModel API key
- SDK:** openai Python package (OpenAI-compatible)
- Context window:** 200,000 tokens
- Pricing:** Quota-based. Off-peak promotional rate through April 2026: 1x quota

4. TECH STACK

Backend

Layer	Technology	Why
Language	Python 3.11+	Most compatible with AI/data tooling
API Framework	FastAPI	Fast, async, auto-generates API docs at /docs
LLM	GLM-5.1 via Z.ai API	OpenAI-compatible, MIT license, strong reasoning
Web Scraping	httpx + BeautifulSoup4	Async HTTP requests, clean HTML parsing
Env Management	python-dotenv	API keys stored safely in .env file
Server	Uvicorn	ASGI server for FastAPI

Frontend

Layer	Technology	Why
Framework	Next.js 14 (App Router)	Matches existing Datawithusman stack
Styling	Tailwind CSS	Fast to build, no separate CSS files needed
HTTP Client	fetch API (built-in)	No extra dependencies for API calls
State	React useState / useReducer	No need for Redux at this scale
Deployment	Netlify or Vercel	Free tier, zero-config deploys

5. FILE STRUCTURE

Backend (Python FastAPI):

```

deallens-backend/
├── main.py           # FastAPI app, all routes
├── analyzer.py       # GLM-5.1 call + prompt logic
├── scraper.py        # Website content fetcher
├── models.py         # Pydantic request/response models
├── prompts.py        # All prompt templates
├── utils.py          # Helper functions
├── requirements.txt  # Dependencies
└── .env              # API keys (not committed to git)

```

Frontend (Next.js):

```

deallens-frontend/
├── app/
│   ├── page.tsx      # Main input form
│   └── results/
│       ├── page.tsx  # Analysis results page
│       └── layout.tsx # Root layout
├── components/
│   ├── InputForm.tsx # Company input form
│   ├── ScoreCard.tsx # K Street Fit Score display
│   ├── SectionCard.tsx # Reusable section card
│   └── LoadingState.tsx # Loading animation
├── lib/
│   └── api.ts         # API call functions
└── .env.local         # Backend URL

```

6. BACKEND: FULL API DESIGN

main.py

This is the entry point. One POST endpoint handles everything.

```

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from models import AnalysisRequest, AnalysisResponse
from analyzer import run_analysis
from scraper import scrape_website

app = FastAPI(title="DealLens API", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.post("/analyze", response_model=AnalysisResponse)
async def analyze(request: AnalysisRequest):
    website_text = ""
    if request.website_url:
        website_text = await scrape_website(request.website_url)

    result = await run_analysis(
        company_name=request.company_name,
        website_text=website_text,
        description=request.description
    )
    return result

@app.get("/health")
async def health():
    return {"status": "ok"}

```

models.py

Pydantic models define the input and output shapes.

```
from pydantic import BaseModel
from typing import Optional

class AnalysisRequest(BaseModel):
    company_name: str
    website_url: Optional[str] = None
    description: Optional[str] = None

class FitScore(BaseModel):
    total: int # 0 to 100
    sector_match: int # 0 to 25
    stage_match: int # 0 to 25
    team_quality: int # 0 to 25
    market_size: int # 0 to 25
    verdict: str # "Strong Fit" / "Possible Fit" / "Weak Fit"

class AnalysisResponse(BaseModel):
    company_name: str
    one_liner: str
    sector: str
    stage: str
    problem_solution: str
    target_market: str
    business_model: str
    team_assessment: str
    traction_signals: str
    competitive_landscape: str
    regulatory_notes: str
    red_flags: str
    fit_score: FitScore
    generated_at: str
```

scraper.py

Fetches and cleans website content before sending to GLM-5.1.

```
import httpx
from bs4 import BeautifulSoup

async def scrape_website(url: str) -> str:
    try:
        headers = {"User-Agent": "Mozilla/5.0"}
        async with httpx.AsyncClient(timeout=10) as client:
            response = await client.get(url, headers=headers, follow_redirects=True)

            soup = BeautifulSoup(response.text, "html.parser")

            # Remove nav, footer, scripts, styles
            for tag in soup(["script", "style", "nav", "footer", "header"]):
                tag.decompose()

            text = soup.get_text(separator=" ", strip=True)

            # Cap at 3000 characters to stay within token limits
            return text[:3000]

    except Exception:
        return ""
```

analyzer.py

Core function. Calls GLM-5.1 and parses the structured JSON response.

```
import os
import json
from datetime import datetime
```

```

from openai import AsyncOpenAI
from prompts import build_prompt
from models import AnalysisResponse, FitScore

client = AsyncOpenAI(
    api_key=os.getenv("GLM_API_KEY"),
    base_url="https://open.bigmodel.cn/api/paas/v4/"
)

async def run_analysis(company_name: str, website_text: str, description: str) ->
AnalysisResponse:
    prompt = build_prompt(company_name, website_text, description)

    response = await client.chat.completions.create(
        model="glm-5.1",
        messages=[
            {
                "role": "system",
                "content": "You are a senior venture capital analyst. You produce structured,
factual startup assessments. You respond only in valid JSON. No markdown, no preamble."
            },
            {
                "role": "user",
                "content": prompt
            }
        ],
        temperature=0.3,
        max_tokens=2000
    )

    raw = response.choices[0].message.content.strip()

    # Strip any accidental markdown fences
    if raw.startswith("```"):
        raw = raw.split("```")[1]
        if raw.startswith("json"):
            raw = raw[4:]

    data = json.loads(raw)

    fit = FitScore(
        total=data["fit_score"]["total"],
        sector_match=data["fit_score"]["sector_match"],
        stage_match=data["fit_score"]["stage_match"],
        team_quality=data["fit_score"]["team_quality"],
        market_size=data["fit_score"]["market_size"],
        verdict=data["fit_score"]["verdict"]
    )

    return AnalysisResponse(
        company_name=company_name,
        one_liner=data["one_liner"],
        sector=data["sector"],
        stage=data["stage"],
        problem_solution=data["problem_solution"],
        target_market=data["target_market"],
        business_model=data["business_model"],
        team_assessment=data["team_assessment"],
        traction_signals=data["traction_signals"],
        competitive_landscape=data["competitive_landscape"],
        regulatory_notes=data["regulatory_notes"],
        red_flags=data["red_flags"],
        fit_score=fit,
        generated_at=datetime.utcnow().isoformat()
    )

```

7. PROMPT ENGINEERING

The prompt is the most important part of this tool. It defines the output structure and tells GLM-5.1 exactly how to think about each section. Temperature is set to 0.3 to keep outputs consistent and factual.

prompts.py

```
def build_prompt(company_name: str, website_text: str, description: str) -> str:
    context_parts = []

    if description:
        context_parts.append(f"Analyst notes: {description}")
    if website_text:
        context_parts.append(f"Website content: {website_text}")

    context = "\n\n".join(context_parts) if context_parts else "No additional context
provided."

    return f"""
You are evaluating a startup for K Street Capital, a Washington DC-based early-stage VC firm.

K Street's investment criteria:
- Stage: Seed to Series A
- Valuation range: $5M to $15M
- Sectors: FinTech, HealthTech, CyberSecurity, ClimateTech, AI, EdTech
- Focus: Regulated marketplaces and high-growth technology
- Does NOT invest in: Hardware, Biotech, Medical Devices, CPG, Food and Beverage, Weapons

Company to evaluate: {company_name}

Available context:
{context}

Produce a JSON object with exactly these fields. Be factual. If information is not available,
write "Insufficient public data" for that field. Do not invent facts.

{{
  "one_liner": "One sentence describing what the company does in plain English",
  "sector": "Single sector classification",
  "stage": "Estimated stage based on available signals",
  "problem_solution": "2 to 3 sentences on the problem they solve and how",
  "target_market": "Who they sell to and estimated TAM",
  "business_model": "How they generate revenue",
  "team_assessment": "Founder backgrounds based on available information",
  "traction_signals": "Any revenue, user, or growth data available publicly",
  "competitive_landscape": "Top 3 competitors and how this company differentiates",
  "regulatory_notes": "Any compliance, licensing, or regulatory considerations",
  "red_flags": "Any concerns: weak team, saturated market, regulatory risk, etc. Write None if
no red flags found.",
  "fit_score": {{
    "sector_match": <0-25, score based on K Street sector alignment>,
    "stage_match": <0-25, score based on K Street stage criteria>,
    "team_quality": <0-25, score based on founder signals>,
    "market_size": <0-25, score based on market size and growth potential>,
    "total": <sum of all four scores>,
    "verdict": <"Strong Fit" if total >= 70, "Possible Fit" if 40-69, "Weak Fit" if below 40>
  }}
}}
"""
```

8. FRONTEND: KEY COMPONENTS

InputForm.tsx

Clean three-field form. Company name is required. URL and description are optional but improve output quality.

```
"use client"
import { useState } from "react"

export default function InputForm({ onSubmit, loading }) {
```

```

const [form, setForm] = useState({
  company_name: "",
  website_url: "",
  description: ""
})

return (
  <div className="max-w-xl mx-auto p-6">
    <h1 className="text-2xl font-bold text-slate-800 mb-2">DealLens</h1>
    <p className="text-slate-500 mb-6 text-sm">
      Paste a startup name and we will produce a VC-style investment snapshot in under 3
minutes.
    </p>

    <div className="space-y-4">
      <input
        type="text"
        placeholder="Company name *"
        value={form.company_name}
        onChange={e => setForm({ ...form, company_name: e.target.value })}
        className="w-full border border-slate-200 rounded px-3 py-2 text-sm"
      />
      <input
        type="url"
        placeholder="Website URL (optional)"
        value={form.website_url}
        onChange={e => setForm({ ...form, website_url: e.target.value })}
        className="w-full border border-slate-200 rounded px-3 py-2 text-sm"
      />
      <textarea
        placeholder="Any context you already have (optional)"
        value={form.description}
        onChange={e => setForm({ ...form, description: e.target.value })}
        rows={3}
        className="w-full border border-slate-200 rounded px-3 py-2 text-sm"
      />
      <button
        onClick={() => onSubmit(form)}
        disabled={!form.company_name || loading}
        className="w-full bg-blue-700 text-white py-2 rounded text-sm font-medium
disabled:opacity-40"
      >
        {loading ? "Analyzing..." : "Run Analysis"}
      </button>
    </div>
  </div>
)
}

```

ScoreCard.tsx

Visual display of the K Street Fit Score. Color changes based on verdict.

```

export function ScoreCard({ score }) {
  const color =
    score.verdict === "Strong Fit" ? "bg-green-100 text-green-800 border-green-200" :
    score.verdict === "Possible Fit" ? "bg-yellow-100 text-yellow-800 border-yellow-200" :
    "bg-red-100 text-red-800 border-red-200"

  return (
    <div className={`border rounded-lg p-4 ${color}`}>
      <div className="flex justify-between items-center mb-3">
        <span className="font-bold text-lg">K Street Fit Score</span>
        <span className="text-3xl font-bold">{score.total}/100</span>
      </div>
      <div className="text-sm font-medium mb-3">{score.verdict}</div>
      <div className="grid grid-cols-2 gap-2 text-xs">
        <div>Sector Match: {score.sector_match}/25</div>
        <div>Stage Match: {score.stage_match}/25</div>
      </div>
    </div>
  )
}

```

```
        <div>Team Quality: {score.team_quality}/25</div>
        <div>Market Size: {score.market_size}/25</div>
    </div>
</div>
)
```

9. ENVIRONMENT SETUP

Backend

```
# 1. Clone and set up
git clone https://github.com/datawithusman/deallens
cd deallens/backend

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install fastapi uvicorn httpx beautifulsoup4 openai python-dotenv

# 4. Create .env file
echo "GLM_API_KEY=your_key_here" > .env

# 5. Run the server
uvicorn main:app --reload --port 8000
```

Frontend

```
# 1. Navigate to frontend
cd deallens/frontend

# 2. Install
npm install

# 3. Create env file
echo "NEXT_PUBLIC_API_URL=http://localhost:8000" > .env.local

# 4. Run
npm run dev
```

requirements.txt

```
fastapi==0.111.0
uvicorn==0.29.0
httpx==0.27.0
beautifulsoup4==4.12.3
openai==1.30.1
python-dotenv==1.0.1
pydantic==2.7.1
```

10. K STREET FIT SCORE: SCORING LOGIC

The scoring system mirrors K Street Capital's actual published investment criteria. Each category is scored out of 25 by GLM-5.1 based on available evidence.

Category	Max Score	Scoring Basis
Sector Match	25	FinTech, HealthTech, CyberSecurity, ClimateTech, AI score highest

Stage Match	25	Seed/Series A score highest. Pre-revenue or Series B+ score lower
Team Quality	25	Second-time founders, domain experts, relevant exits score highest
Market Size	25	TAM >\$1B with regulatory tailwinds scores highest

Total Score	Verdict	Meaning
70 to 100	Strong Fit	Worth a first meeting. Fast-track for review.
40 to 69	Possible Fit	Needs deeper research before deciding on a meeting.
0 to 39	Weak Fit	Does not match current fund thesis.

11. README FOR GITHUB

This is what goes in the README.md file in the GitHub repo. This is what K Street will read when they click the link.

```
# DealLens

A VC deal research tool that produces structured investment snapshots in under 3 minutes.

Built for analysts who spend 2-3 hours manually researching every inbound startup before
deciding whether to take a meeting. DealLens cuts that to minutes.

## What it produces

For any startup name or URL, DealLens returns:
- Company one-liner
- Sector and stage classification
- Problem, solution, and target market
- Business model summary
- Team assessment
- Traction signals
- Competitive landscape
- Regulatory notes
- Red flags
- K Street Capital Fit Score (0 to 100)

## Stack

Backend: Python, FastAPI, GLM-5.1 (Z.ai)
Frontend: Next.js, Tailwind CSS

## Setup

See /docs/setup.md for full installation instructions.

## Why I built this

I was researching K Street Capital's portfolio and investment thesis and noticed
the firm manages 85+ companies with a lean team across three investment vehicles.
The analyst role at a firm like that requires moving fast on deal evaluation.
This tool is my answer to that problem.

Built by Muhammad Usman — datawithusman.com
```

12. V2 FEATURES (AFTER V1 SHIPS)

These are not in scope for the first version. Build them after the core works.

- PDF pitch deck upload: extract text from a pitch deck and include it in the analysis
- Founder LinkedIn scraping: auto-pull founder background from LinkedIn URLs
- Crunchbase integration: pull funding history via Crunchbase API
- Export to PDF: one-click export of the analysis as a clean PDF report
- Batch mode: paste 10 company names and run all analyses in sequence
- History: save past analyses locally so you can reference them later

13. WHAT MAKES THIS DIFFERENT FROM GENERIC AI TOOLS

ChatGPT and similar tools can produce a basic company summary. DealLens is different in three ways.

- The output format mirrors what a VC analyst actually needs. It is not a general summary, it is a structured first-cut memo with a fit score tied to a specific fund's criteria.
- The K Street Fit Score is calculated against real published criteria: sector focus, stage range, valuation range, and investment thesis. A generic AI tool has none of this context.
- It is built as a tool, not a chat interface. You paste a name, you get a structured report. No prompt engineering required from the user.

The GitHub repo shows Paige and her team that the person applying for this role already understands their specific workflow and has built something that would save them real time from day one.